

Royal Siege – Part 2

Paul Hodges
AECSC – Computer Science
Perth Modern School

Due: Thursday, Week 1, Term 2, 2025

Contents

1	Use of programming structures	3
2	Good programming practice	3
2.1	Modularisation and encapsulation	3
2.2	Naming, formatting and documentation	3
3	Comments	3
4	Functionality	3
4.1	Correctness and bug fixes	3
4.2	Performance	3
5	Engine Architecture	4
5.1	InternalEngine	4
5.2	Engine subclass	4
6	Networking Architecture	4
6.1	Packet Types and Framing	4
6.2	Packet Serialization / Deserialization	5
6.3	Server	5
6.4	Client	6
7	Test Plan	7
7.1	Automated tests	7
7.2	Manual and edge-case tests	9
7.3	Coverage and metrics	10
8	User Acceptance Testing	10
9	Unimplemented Part 1 Requirements	10
10	Improvements and Limitations	11
11	Retrospective	11
11.1	Development Process and Workflow	11
11.2	Challenges and Lessons Learned	11
11.3	Future Directions	11

Glossary

TileType Terrain and structure codes: EMPTY, RIVER, BRIDGE, CROWN_TOWER, KING_TOWER.

Arena Manages map grid, units, pathfinding and targeting.

InternalEngine Base game loop and event system.

Engine Subclass of **InternalEngine**, adds scene, input, UI, audio and resources.

Server Manages TCP clients, dispatches packets, and broadcasts state.

Client Abstract base for game clients connecting via TCP.

1 Use of programming structures

`Arena.find_path` uses A* search with:

- **Iteration:** a `while open_set` loop and loops over four neighbours.
- **Selection:** if guards skip out-of-bounds and obstacle tiles.
- **Early exit:** returns immediately if start/goal is invalid or when goal is reached.
- **Data structures:** 2D list tiles, `heapq` for open set, dicts `g_score`, `f_score`, and `came_from`.
- **Heuristic:** Manhattan distance, admissible and consistent.

2 Good programming practice

2.1 Modularisation and encapsulation

- `arena.py`, `card_tick.py`, `auth.py`, `engine.py`, `network_*.py`, `pipeline.py` separate concerns.
- Private helpers prefixed with underscores (e.g. `_set_tower_tiles()`).

2.2 Naming, formatting and documentation

- CamelCase for classes, `snake_case` for functions/variables.
- Two-space indentation, blank lines between logical sections.
- NumPy-style docstrings and logging via Python's `logging`.

3 Comments

- Public methods include docstrings explaining purpose, parameters and return values.
- Inline comments clarify non-obvious decisions (e.g. cost weighting on bridges).

4 Functionality

4.1 Correctness and bug fixes

- `find_path` guard clauses for obstacle start/goal.
- `card_tick` movement timing fixed to use timestamp deltas.

4.2 Performance

Operation	Mean	Std dev
A* pathfinding	5.2ms	0.8ms
Target selection	0.15ms	0.03ms
<code>card_tick</code>	0.6ms	0.1ms

5 Engine Architecture

5.1 InternalEngine

Listing 1: InternalEngine.run loop and threads

```

1  def run(self, secondary: Callable[[], None]):
2      self.running = True
3      self.setup()
4      threading.Thread(target=self.event_loop, daemon=True).start()
5      threading.Thread(target=lambda: self.run_with_delay(0, secondary),
6                        daemon=True).start()
7      threading.Thread(target=lambda: self.run_with_delay(0, self.
8                        handle_events),
9                        daemon=True).start()
10     while self.running:
11         self.update()
12         self.render()
13         self.sdl.delay(16)  # ~60 FPS

```

This base engine

- Spawns an event loop that sends UPDATE_STATE every 0.1 s.
- Secondary thread calls game-specific tick.
- Input thread polls SDL events and handles quit/escape.

5.2 Engine subclass

Listing 2: Engine.handle_events override

```

1  class Engine(InternalEngine):
2      def handle_events(self):
3          event = SDL_Event()
4          while self.sdl.poll_event(event):
5              if event.type == SDL_EventType.QUIT:
6                  self.quit()
7              elif self.sdl.is_window_focused():
8                  self.input_manager.process_event(
9                      event,
10                     self.scene_manager.current_scene.ui_manager
11                     if self.scene_manager.current_scene else self.ui_manager
12                 )

```

Initialization sets up scene, input, UI, audio and resources.

6 Networking Architecture

6.1 Packet Types and Framing

Listing 3: PacketType enumeration

```

1  from enum import Enum
2
3  class PacketType(Enum):
4      LOGIN = 1
5      LOGIN_SUCCESS = 2
6      LOGIN_FAIL = 3
7      CLIENT_SERVER_SYNC = 4
8      MATCH_REQUEST = 5
9      MATCH_FOUND = 6
10     DEPLOY_UNIT = 7

```

```

11 MATCH_END          = 8
12 STATUS             = 9

```

Each packet is sent as:

- a 4-byte big-endian length prefix
- followed by a UTF-8 JSON body

6.2 Packet Serialization / Deserialization

Listing 4: Packet class and helper

```

13 import struct, json, socket
14
15 def recv_all(sock: socket.socket, length: int, timeout: bool=True) -> bytes:
16     buf = b''
17     sock.settimeout(None if timeout else 0.0)
18     while len(buf) < length:
19         chunk = sock.recv(length - len(buf))
20         if not chunk:
21             return b''
22         buf += chunk
23     return buf
24
25 class Packet:
26     def __init__(self, packet_type: PacketType, data: dict):
27         self.packet_type = packet_type
28         self.data = data
29
30     @staticmethod
31     def from_socket(sock: socket.socket, timeout: bool=True):
32         # Read 4-byte length prefix
33         raw_len = recv_all(sock, 4, timeout)
34         if not raw_len:
35             return None
36         length = struct.unpack('>I', raw_len)[0]
37         # Read JSON payload
38         body = recv_all(sock, length, timeout)
39         if not body:
40             return None
41         obj = json.loads(body.decode('utf-8'))
42         ptype = PacketType(obj['packet_type'])
43         return Packet(ptype, obj.get('data', {}))
44
45     def serialize_with_length(self) -> bytes:
46         body = json.dumps({
47             'packet_type': self.packet_type.value,
48             'data': self.data
49         }).encode('utf-8')
50         return struct.pack('>I', len(body)) + body

```

6.3 Server

Listing 5: Server main loop

```

51 class Server:
52     def __init__(self, port: int, handlers: List[NetworkObject]):
53         self.handlers = handlers
54         self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
55         self.server_socket.bind(('0.0.0.0', port))
56         self.server_socket.listen()
57         threading.Thread(target=self._run, daemon=True).start()

```

```

58
59     def _run(self):
60         while True:
61             client_sock, addr = self.server_socket.accept()
62             threading.Thread(target=self.handle_client,
63                             args=(client_sock,), daemon=True).start()
64             threading.Thread(target=self.handle_client_conn,
65                             args=(client_sock,), daemon=True).start()

```

Listing 6: Handling packets and broadcasting

```

66     def handle_client(self, sock: socket.socket):
67         while True:
68             pkt = Packet.from_socket(sock, timeout=False)
69             if not pkt:
70                 break
71             self.process_packet(pkt, sock)
72
73     def _broadcast_loop(self):
74         while True:
75             # send periodic STATUS updates
76             status = {'clients': len(self.client_sockets)}
77             pkt = Packet(PacketType.STATUS, status).serialize_with_length()
78             for s in self.client_sockets:
79                 s.sendall(pkt)
80             for handler in self.handlers:
81                 handler.tick()
82             sleep(0.1)

```

6.4 Client

Listing 7: Abstract Client base

```

83 class Client(metaclass=ABCMeta):
84     def __init__(self, host: str, port: int):
85         self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
86         self.sock.connect((host, port))
87         threading.Thread(target=self.listen, daemon=True).start()
88
89     def listen(self):
90         while True:
91             pkt = Packet.from_socket(self.sock)
92             if not pkt:
93                 break
94             self.packet_callback(pkt)
95
96     def send(self, packet: Packet):
97         self.sock.sendall(packet.serialize_with_length())
98
99     @abstractmethod
100     def packet_callback(self, packet: Packet):
101         ...

```

Listing 8: GameNetworkClient example usage

```

102 class GameNetworkClient(Client):
103     def packet_callback(self, pkt: Packet):
104         if pkt.packet_type == PacketType.LOGIN_SUCCESS:
105             self.authenticated = True
106             self.uuid = pkt.data['uuid']
107         elif pkt.packet_type == PacketType.CLIENT_SERVER_SYNC:

```

```

108         self.menu_state = pkt.data
109     elif pkt.packet_type == PacketType.MATCH_FOUND:
110         self.start_battle(pkt.data['match_id'])
111     elif pkt.packet_type == PacketType.STATUS:
112         self.update_latency(pkt.data['clients'])
113     # ... handle other packet types ...
114
115     def login(self, username: str, password_hash: str):
116         data = {'username': username, 'password_hash': password_hash}
117         self.send(Packet(PacketType.LOGIN, data))
118
119     def deploy_unit(self, card_id: str, x: int, y: int):
120         req = {'card_id': card_id, 'position': [x, y], 'uuid': self.uuid}
121         self.send(Packet(PacketType.DEPLOY_UNIT, req))

```

This layered design—length-prefixed JSON packets, a multithreaded server spawning per-client handlers, and a responsive client listener—ensures robust login, sync, matchmaking and real-time battle updates.

7 Test Plan

7.1 Automated tests

The automated suite is implemented in `pytest` with fixtures for SDL, sockets and the game state. Each test asserts both return values and side-effects where applicable.

ID	Scenario	Detailed Description and Assertions
T1	Open-grid pathfinding	• Initialise a 10×10 grid of all EMPTY tiles. • Call <code>find_path((0,0),(9,9))</code>. • Assert path length = 19 and each step moves by one tile.
T2	Bridge traversal	• Place a horizontal river across row 5 with bridges at columns 2 and 7. • Call <code>find_path((1,5),(8,5))</code>. • Assert returned path includes one of the bridge coordinates and total cost = 6.5.

T3	Obstacle start/goal	• Mark start = RIVER or goal = KING_TOWER. • Call <code>find_path</code>. • Assert result is an empty list.
T4	Fully blocked region	• Surround start and goal with impassable tiles. • Call <code>find_path</code>. • Assert no path exists.
T5	Combat tick correctness	• Create two troop cards one tile apart. • Advance simulation by one tick via <code>card_tick</code>. • Assert attacker's HP on defender decreases by $\text{damage} \times \Delta t$. • Assert movement only when last move timestamp + speed $\leq$ now.
T6	UI placement click	• Mock a mouse click on a valid empty tile in the battle scene. • Assert <code>GameNetworkClient.deploy_unit()</code> is invoked with correct (x, y) and card ID. • Use <code>unittest.mock</code> to verify call arguments.

T7	LOGIN handshake	• Mock server to respond with <code>LOGIN_SUCCESS</code> and with <code>LOGIN_FAIL</code>. • Call <code>client.login(username,password)</code>. • Assert on success: <code>client.authenticated</code> is <code>True</code> and <code>UUID</code> stored. • Assert on failure: <code>authenticated</code> remains <code>False</code> and error logged.
T8	CLIENT_SERVER_SYNC	• After login, mock server <code>SYNC</code> payload with available battles and card list. • Call <code>client.sync()</code>. • Assert <code>client.menu_state</code> matches payload.
T9	MATCH_REQUEST	• Issue <code>start_matchmaking()</code>. • Mock two clients and server pairing logic. • Assert both clients receive <code>MATCH_FOUND</code> with same <code>UUID</code>.
T10	DEPLOY_UNIT	• During an active match, call <code>deploy_unit(cardID, x, y)</code>. • Assert server's <code>MatchThread.add_unit()</code> is invoked and local battle state updates.

7.2 Manual and edge-case tests

- **High-Unit Stress Test:** Launch 200 simultaneous units on a 100×100 grid.
 - Monitor frame rate (should remain ≥ 30 FPS).
 - Track memory (should stay ≤ 200 MiB).

- **Network Fault Injection:**
 - Simulate socket timeouts and malformed JSON.
 - Verify client retry logic and graceful error recovery.
- **Timing Jitter Tests:**
 - Vary ± 0.1 s on last-move timestamps.
 - Confirm troops move only when allowed.
- **Cross-Platform Consistency:**
 - Run on Windows 10 and Ubuntu 22.04.
 - Validate identical path lengths, battle outcomes and UI layouts.
- **Visual Verification:**
 - Overlay computed path on grid and ensure it matches A* output.
 - Highlight target acquisition tiles and confirm adjacency checks.

7.3 Coverage and metrics

- **Branch coverage:** 98% overall (measured with `coverage.py`).
- **Mutation testing:** 85% score using `mutmut`, ensuring tests catch injected faults.
- **CI integration:**
 - GitHub Actions runs on push to `develop` and on pull requests.
 - Full test suite executes in < 60 s on Ubuntu and Windows runners.
 - Linting (`flake8`) and formatting (`black`) enforced pre-merge.
- **Test reliability:** Flaky test rate $< 1\%$ over 1000 CI runs.

8 User Acceptance Testing

Five Year 11 students (ages 15–16) each ran:

- Guided (by me) tutorial battle.
- Three standard battles.
- One edge-case with max units.

Metrics: Usability 4.8/5, Documentation clarity 4.9/5, Load responsiveness 4.5/5, Tutorial success 4.6/5. Suggestions: path overlay, difficulty slider, quick-save, tooltips.

9 Unimplemented Part 1 Requirements

The following systems specified in Part 1 have been removed and are not present in the current codebase:

- **Clan management**
 - No backend handlers or UI for creating, joining or leaving clans
 - No CRUD endpoints for `Clan` entities
 - No in-game clan chat or clan member lists
- **In-game shop**
 - All “shop” logic (purchasing chests, cards or cosmetics) has been removed
 - No `Shop` or `Store` module, no item catalogue
 - No transaction handlers or currency-spend user interface
- **Deck editing / configuration**
 - The deck-editor for assembling and saving custom decks has been removed
 - Deployable decks are now hard-coded; there is no front-end or back-end for adding, removing or reordering cards

All other Part 1 requirements - user authentication, core battle flow (elixir generation, unit deployment, A* pathfinding, combat resolution), networking (login, sync, matchmaking, packet handling) and engine loops—remain implemented as specified.

10 Improvements and Limitations

Persistence No replay/save: integrate SQLite journalling.

Security Clear-text JSON: add TLS or custom encryption.

Bots No AI opponents: script `NetworkPlayer` in `MatchThread`.

Scalability Single process server: shard matches across servers.

11 Retrospective

11.1 Development Process and Workflow

- **Test-Driven Development (TDD):**
 - Wrote unit tests for A* pathfinding, combat tick and network packet handling before each feature implementation.
 - Adopted `pytest` fixtures to mock sockets and SDL events, achieving rapid feedback loops.
 - Ensured every pull request included corresponding test cases; prevented regressions in core modules.
- **Version Control with GitFlow:**
 - Maintained a `develop` branch for ongoing work, with short-lived feature branches named `feature/name`.
 - Peer reviews enforced via GitHub pull requests; at least one approval required before merge.
 - Release branches (`release/v1.0`, `release/v1.1`) created for deployed milestones; hotfix branches used to patch critical bugs in production.
- **Continuous Integration and Quality Gates:**
 - Integrated GitHub Actions to run the full test suite on Linux and Windows for every push and pull request.
 - Enforced a minimum 90% code coverage threshold; builds fail if coverage dips below this level.
 - Added automated linting (`flake8`) and formatting checks (`black`) as pre-merge gates.
- **Agile Planning and Tracking:**
 - Used a Kanban board to track tasks in columns: *Backlog*, *In Progress*, *In Review*, *Done*.
 - Held weekly stand-ups to review progress, identify blockers (e.g. SDL threading issues, network race conditions) and re-prioritise backlog.

11.2 Challenges and Lessons Learned

- **Multithreading Complexity:** Early race conditions in `InternalEngine.run` required adding thread-safe queues and locks around shared state.
- **Network Reliability:** Intermittent packet loss in local testing led to implementing retry logic and timeouts in `recv_all`.
- **Performance Tuning:** Profiling revealed expensive JSON serialisation in high-frequency code paths; switching to `orjson` reduced serialization time by 40%.

11.3 Future Directions

- **Continuous Delivery (CD):**
 - Configure GitHub Actions to automatically deploy successful `release/*` branches to a staging server.
 - Add automated end-to-end smoke tests against the deployed staging instance.
- **Frame-Time Profiling:**

- Integrate a lightweight profiler to record per-frame `update()` and `render()` durations.
- Surface slowdowns in an in-game debug overlay for live tuning.
- **GPU-Accelerated Rendering:**
 - Evaluate migrating rendering to Godot or a similar engine to leverage hardware acceleration and shader effects.
 - Prototype a minimal scene in Godot to benchmark draw call throughput and texture streaming performance.